DAY 3 Cleaning the Text

Project: Predicting Physical and Electronic Properties of Crystalline Solids from their Text Descriptions

Architecture block: 2. Text Preparation (part 1: clean text)

Date: Thursday, 22 Oct 2026

GOAL OF DAY 3

Clean the crystal descriptions the same way the paper authors did, and save the clean files.

What we finished today:

- ✓ Copied the authors' repo to get their stopword lists
- ✓ Wrote preprocess.py: [NUM], [ANG] and stopword removal
- ✓ Tested it on 2,000 real crystals
- ✓ Found a problem: element symbols (O, Mg, Si) get deleted
- ✓ Added Version B that protects element symbols
- ✓ Wrote run_preprocess.py and cleaned ALL the data
- ✓ Saved results in notes.md and pushed to GitHub

NOTE: Day 1 + 2 + 3 done = 3 of 20 days. The cleaned files are in data/processed/A/. Next: tokenizing (Day 4).

1. Why clean the text?

Block 2 of the architecture

- The raw description is long and full of filler words (the, is, of ...). The model has a limit of 888 tokens, so every useless word wastes space.
- Also, exact bond distances like 2.04-2.36 A are hard for a language model to use as words. The authors replace them with a simple tag.

The authors' recipe (3 steps):

- Bond distances -> [NUM] e.g. 2.04-2.36 Å becomes [NUM]
- Bond angles -> [ANG] e.g. 120.5° becomes [ANG]
- Remove stopwords (using their lists, but keep one..nine)

([CLS] is added later when we tokenize, on Day 4.)

BEFORE: Charges and subscripts are shown plain here to keep it simple. Li+ is bonded in a 5-coordinate geometry to five O atoms. There are a spread of Li-O bond distances ranging from 2.04-2.36 Å. Fe is bonded to six O atoms to form distorted FeO6 octahedra that share corners with four equivalent AsO4 tetrahedra.

AFTER: Li+ bonded 5-coordinate geometry five O atoms. spread Li-O bond distances ranging [NUM]. Fe bonded six O atoms form distorted FeO6 octahedra share corners four equivalent AsO4 tetrahedra.

32 Copy the authors' repo (for their stopword lists)

```
> git clone https://github.com/vertaix/LLM-Prop.git data\LLM-Prop-ref
> dir data\LLM-Prop-ref\stopwords\en /b
```

- About 70 .txt lists (nltk, spacy, gensim ...) get merged into ONE stopword set. The repo is read-only: we write our OWN code (Git ignores data/).

2. How preprocess.py works

step 33: my second real script

33 Put the file in src and run it

```
> move "%USERPROFILE%\Downloads\preprocess.py" src\preprocess.py
> python src/preprocess.py
```

- The test reads only 2,000 crystals from validation.csv, so it takes a few seconds.

The 4 functions inside:

`load_stopwords()`	merge all .txt lists into one set; drop numbers
`replace_num_and_ang()`	distances -> [NUM], angles -> [ANG]
`remove_stopwords()`	delete words found in the stopword set
`preprocess_text()`	runs the steps in the right order

Two small but important details:

- ORDER: replace distances/angles FIRST, then remove stopwords. This way the tags [NUM] and [ANG] are safe.
- one, two, ... nine are NOT removed. They tell how many neighbours an atom has (bonded to five O atoms).

The pattern for a distance (regex), in words:

- a number + maybe a range like -2.41 + maybe a space + the Angstrom sign

```
> \d+(\.\d+)?(-\d+(\.\d+)?)?\s*[Angstrom sign]   ->   [NUM]
```

(angles use the degree sign instead; also the word 'degrees')

TIP: A regex is a search pattern. \d = a digit, + = one or more, ? = optional, \s = a space.

3. Results on real crystals

2,000 crystals from validation.csv

`Stopwords loaded`	2,365 unique words from 68 files
`Words before`	784,394
`Words after`	482,987 (38.4 percent removed)
`[NUM] replaced`	15,617 times
`[ANG] replaced`	5,735 times
`Å or ° left over`	0 texts (the patterns work!)

Most removed words:

- to (31,925) is (26,862) a (24,714) and (22,246) the (21,882) with (20,469) In (20,031) in (16,847)

Real example (crystal 1), the end of the cleaned text:

AFTER: Li2FeCAsO7 crystallizes monoclinic P2_1/m space group. ... spread Fe-O bond distances ranging [NUM]. ... one shorter ([NUM]) two ([NUM]) C-O bond lengths.

✓ The sentences are shorter, but all the chemistry words are still there. Good!

REMEMBER: Words are not tokens. The model counts TOKENS (pieces of words), and one word can be several tokens. We measure real tokens on Day 4 with the tokenizer.

4. The problem I found

stopword lists delete chemistry words

- The last line of the test printed which tiny words were removed. Some are chemical ELEMENT symbols:

`o (oxygen)`	removed 2,271 times
`mg (magnesium)`	removed 1,469 times
`si (silicon)`	removed 521 times
`al (aluminium)`	removed 390 times
`co (cobalt)`	removed 322 times

- Why: some of the 68 lists contain tiny abbreviations like 'o', 'mg', 'si', 'al', 'co'. The code compares lowercase words, so the elements get removed together with 'the' and 'is'.
- This is how I read the authors' method, so Version A = authors' method, problem included.

My fix = Version B:

- Version A: authors' method (stopword lists as they are)
- Version B: same, but 110 element symbols are PROTECTED

✓ On 2,000 crystals, B keeps 10,125 more words than A (493,112 vs 482,987). That is about 2 percent more text, all of it chemistry.

- Not protected: In, As, At, Be, No, He, I, Am. They are also normal English words ('in the first site'), so protecting them would keep lots of real stopwords.

IDEA: This becomes MY experiment in the ablation block: train with A and with B, compare the scores. It fits my problem statement: can carefully designed text help small models?

5. Clean the full dataset

steps 35 to 37

35 Update preprocess.py, add run_preprocess.py

```
> move /Y "%USERPROFILE%\Downloads\preprocess.py" src\preprocess.py
> move "%USERPROFILE%\Downloads\run_preprocess.py" src\run_preprocess.py
> dir src
```

- /Y = replace the old file without asking. dir src should show 3 files: explore_data.py, preprocess.py, run_preprocess.py

36 Quick test again

```
> python src/preprocess.py
```

✓ New line appeared: Version B (protect elements) keeps 10125 more words ... = I have the new version.

37 Clean ALL the data (Version A)

```
> python src/run_preprocess.py
```

- It took only about 30 seconds (progress bar for each file).

train	125,098 -> 125,065 rows (dropped 33)
validation	9,945 -> 9,940 rows (dropped 5)
test	11,531 -> 11,529 rows (dropped 2)

- Rows dropped = rows with a missing label (volume or direct/indirect).

✓ Median words per text: 224 before -> 139 after cleaning.

Saved in: data\processed\A\train.csv, validation.csv, test.csv

Columns: material_id, description (clean), band_gap, volume, is_gap_direct (now 1 or 0)

NOTE: Version B is made later with python src/run_preprocess.py B (only when we do the ablation).

6. Save and what is next

step 38 and Day 4 plan

38 Save Day 3 to GitHub

```
> echo Day 3 results: run_preprocess.py cleaned all splits>> notes.md
> echo Median words per text fell from 224 to 139>> notes.md
> git add notes.md src/preprocess.py src/run_preprocess.py
> git commit -m "Day 3: preprocess full dataset, add Version B option"
> git push
```

output: `git status: up to date, nothing to commit = done!`

Small problems and fixes today:

✓ Which file to replace? use move /Y to overwrite the old preprocess.py

✓ (venv) missing? run venv\Scripts\activate in the LLM_PROP folder

Files in my project now:

`src/explore_data.py`	(Day 2)
`src/preprocess.py`	(Day 3: the cleaning functions)
`src/run_preprocess.py`	(Day 3: cleans all the files)
`data/processed/A/`	(clean data, only on my laptop)

Next: Day 4 (Block 2, part 2)

- Add [CLS] at the start of each text.
- Tokenize with the authors' T5 tokenizer (in data\LLM-Prop-ref\tokenizers), max 888 tokens.
- Scale the labels with z-score (regression: band gap, volume).
- Make a Dataset + DataLoader. Then Block 3: the model.

3 days done. Keep the streak going! ✳ ✳